>_ MCMP113

Introduction to Basic Algorithms and Programming

Week 8

Midterm Review & Practice Exam

```
> dotnet run
Loading weeks 1-7 ... done
Practice questions: 12
Practice exam: 90 min
> ready?_
```

Weeks 1–7 condensed · 12 practice questions with answers · exam strategy

Asst. Prof. Hasan Oğuz, PhD
Istanbul Okan University · Vocational School
2026–2027 Fall Semester

>_ TODAY

What We Will Do in This Session

Review

All seven weeks condensed into nine slides: algorithms, flowcharts, types, operators, decisions, loops.

Practise

12 exam-style questions: multiple choice, predict, trace, find the bug, flowchart, write a program.

Plan

A 90-minute time plan and a method for each question type.

Self-test

Take the full practice midterm in the notes under exam conditions, then check it.

Session timeline (approx.)



Rule for today: answer every question on paper BEFORE the answer slide appears. Reading answers feels like learning; producing them is what the exam measures.

>_ EXAM LOGISTICS

The Midterm at a Glance

When & where

Date, room and duration: announced on the course group.

Closed book

No notes, no phones, no laptops. Only what is in your head.

Bring

Student ID card, pen. Pencil and eraser for flowcharts.

Weight

Midterm 40 % · Final 60 % of the course grade.

On paper

You write C# by hand: no compiler, no IntelliSense. Practise that way.

Question types

Multiple choice, predict output, trace table, flowchart, find bugs, write a program.

>_

Announcements

Official date, room and any seating plan are announced in class and on the course page.

What the Midterm Covers: Weeks 1–7

Week	Topic	You must be able to …
1	Algorithms & problem solving	name the 5 properties; write pseudocode; build test cases and trace tables
2	Flowcharts	draw with the correct symbols; convert flowchart ⇄ pseudocode
3	Variables, data types, constants	choose a type; declare/initialise; Parse input; cast; format output ({x:F2})
4	Operators	evaluate expressions with precedence, / and %, ++/--, && || !, short-circuit
5	if, else-if, switch	write selection logic; validate input with TryParse; order conditions correctly
6	for loop	count, accumulate, nest loops; predict patterns; use break/continue
7	while, do-while	sentinel and validation loops; digit loops (% 10, / 10); choose the right loop



Not on the midterm: arrays (Week 9 on) and methods/functions (Week 12 on). Do not use them in your answers.

Seven Weeks in One Picture

Sequence

Weeks 1–4

```
Console.Write("r = ");
double r = double.Parse(
    Console.ReadLine());
double a = Math.PI * r * r;
Console.WriteLine($"{a:F2}");
```

Selection

Week 5

```
if (grade >= 50)
{
    Console.WriteLine("PASS");
}
else
{
    Console.WriteLine("FAIL");
}
```

Repetition

Weeks 6–7

```
int sum = 0;
for (int i = 1; i <= n; i++)
{
    sum += i;
}
Console.WriteLine(sum);
```

Böhm–Jacopini (Week 1): sequence, selection and repetition are enough for every algorithm. Most exam questions combine two of them, e.g. a decision inside a loop.



PART 1

Condensed Review

Weeks 1–7 in nine slides: the facts you must have at your fingertips

Algorithm Properties & Problem-Solving Steps

Property	Meaning	Violated by ...
Finiteness	stops after a finite number of steps	"Lather, rinse, repeat."
Definiteness	every step precise, unambiguous	"Add salt to taste."
Input	zero or more well-specified inputs	"Take the bread." (which?)
Output	at least one result	nothing is ever printed
Effectiveness	each step can really be done	"Pick the best team."

Problem-solving cycle

- **Understand** input, output, constraints
- **Design** pseudocode / flowchart
- **Implement** C# code
- **Test** normal · boundary · invalid
- **Document** names, comments

Trace table = one column per variable (plus output), one row per change or per loop pass. Write the values the variables have AFTER each step.

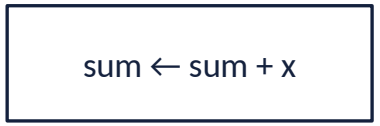
Flowchart Symbols and Drawing Rules



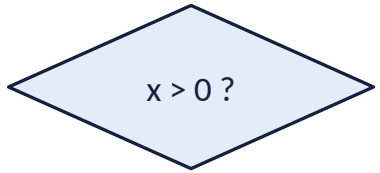
Terminal



Input / Output



Process



Decision



Predefined process



Connector

Flow line: always with an arrow head

Rules checked in the exam

- exactly one Start and one End
- flow top → bottom, arrows on every line
- every decision: two exits labelled yes / no
- every path reaches End
- a loop needs an update (else infinite)
- short, precise text in boxes (← assigns)

Pseudocode ⇄ C#: the Translation Table

Pseudocode	C#
READ x	int x = int.Parse(Console.ReadLine());
PRINT "Sum:", s	Console.WriteLine($"Sum: {s}");
x ← x + 1	x = x + 1; x += 1; x++;
a = b a ≠ b	a == b a != b
a mod b	a % b
IF c THEN … ELSE … END IF	if (c) { … } else { … }
WHILE c … END WHILE	while (c) { … }
FOR i ← 1 TO n … END FOR	for (int i = 1; i <= n; i++) { … }
AND OR NOT	&& || !

← assigns
= compares

In C# the symbols change:
= assigns
== compares

if (x = 5) does not compile for an int x: CS0029, cannot convert int to bool.

Data Types You Must Know

Type	Size	Example literal	Use it for
`int`	4 B	`42`	whole numbers, about ±2.1 billion
`long`	8 B	`6227020800L`	large whole numbers (factorials)
`double`	8 B	`72.5`	measurements, scientific values
`float`	4 B	`2.5f`	rarely: less precise than double
`decimal`	16 B	`19.99m`	money (exact decimal digits)
`char`	2 B	`'A'`	one character (single quotes)
`string`	—	`"Hello"`	text (double quotes)
`bool`	1 B	`true`	yes/no results of conditions

Declarations

```
const double VatRate = 0.20;
var count = 0;   // int
decimal price = 19.99m;
char grade = 'A';
```



decimal price = 19.99; does not compile: 19.99 is a double literal. Add m.

Reading Input, Converting, Formatting

Program.cs (excerpt)

```
Console.Write("n = ");
bool ok = int.TryParse(Console.ReadLine(), out int n);
if (!ok)
{
    Console.WriteLine($"Not a number (n = {n})");
}
double avg = (double)7 / 2;        // 3.5
Console.WriteLine($"{avg:F2} {(int)7.9}");
```

Console

```
n = 12a
Not a number (n = 0)
3.50 7
```

Expression	Value
`7 / 2`	`3`
`7 / 2.0`	`3.5`
`(double)7 / 2`	`3.5`
`(double)(7 / 2)`	`3`
`(int)7.9`	`7`
`int.Parse("7.5")`	`crash`
`int.TryParse("x", out v)`	`false, v = 0`

(double)(7 / 2) is 3 because the integer division happens first; printed as 3.

Operator Precedence: Evaluate Like the Compiler

Level	Operators
1 (highest)	`x++  x--  (postfix)`
2	`!  -x  ++x  --x  (type)x`
3	`*  /  %`
4	`+  -`
5	`<  >  <=  >=`
6	`==  !=`
7	`&&`
8	`||`
9	`?:`
10 (lowest)	`=  +=  -=  *=  /=  %=`

Step by step (one operator per line)

```
2 + 3 * 4 % 5 > 3 && !false
= 2 + 12 % 5 > 3 && !false
= 2 + 2 > 3 && !false
= 4 > 3 && !false
= true && !false
= true && true
= true
```

Expression	Result
`-17 % 5`	`-2`
`10 - 4 / 2 * 3`	`4`
`b != 0 && a / b > 2   (b = 0)`	`false, no crash`

if / else-if / switch: Three Ways to Choose

else-if ladder

```
if (score >= 85)
{
    letter = "BA";
}
else if (score >= 70)
{
    letter = "CB";
}
else if (score >= 50)
{
    letter = "DD";
}
else
{
    letter = "FF";
}
```

switch statement

```
switch (day)
{
    case 6:
    case 7:
        kind = "Weekend";
        break;
    default:
        kind = "Weekday";
        break;
}
```

switch expression

```
letter = score switch
{
    >= 85 => "BA",
    >= 70 => "CB",
    >= 50 => "DD",
    _     => "FF"
};
```

conditional ?:

```
string parity =
    n % 2 == 0
    ? "even" : "odd";
```

for, while, do-while: Which Loop When?

for

```
for (int i = 1; i <= 3; i++)
{
    Console.Write(i + " ");
}
```

Use when: Number of passes known in advance (counter).

Body runs: 0 or more

Output: `1 2 3`

while

```
int i = 1;
while (i <= 3)
{
    Console.Write(i + " ");
    i++;
}
```

Use when: Repeat while a condition holds; sentinel input.

Body runs: 0 or more (pre-test)

Output: `1 2 3`

do-while

```
int i = 1;
do
{
    Console.Write(i + " ");
    i++;
} while (i <= 3);
```

Use when: Body must run first: menus, input validation.

Body runs: at least 1 (post-test)

Output: `1 2 3`

Five Loop Patterns That Solve Most Exam Problems

1 Accumulator

```
sum ← 0
FOR i ← 1 TO n
  READ x
  sum ← sum + x
END FOR
```

2 Best so far

```
READ x
max ← x
FOR i ← 2 TO n
  READ x
  IF x > max
  THEN max ← x
  END IF
END FOR
```

3 Sentinel

```
READ x
WHILE x ≠ -1
  use x
  READ x
END WHILE
```

4 Validation

```
REPEAT
  READ n
UNTIL n ≥ 0

C#: do { … }
while (n < 0);
```

5 Digit loop

```
WHILE n > 0
  d ← n mod 10
  n ← ⌊n / 10⌋
END WHILE
```

Pseudocode skeletons. REPEAT … UNTIL is the do-while shape: C# repeats WHILE the input is bad (condition negated).



Exam habit: before writing code, name the pattern ("sentinel + accumulator + best so far"). The skeleton then writes itself.



PART 2

Practice I: Read & Predict

Multiple choice · predict the output · trace tables. Paper first, then the answer slide.

MULTIPLE CHOICE · 2 min

Quick-Fire Multiple Choice

Q1 "Keep adding 1 to x." Which property fails?

- A) Definiteness
- B) Finiteness
- C) Input
- D) Output

Q2 Which flowchart symbol is used for READ x?

- A) Rectangle
- B) Diamond
- C) Parallelogram
- D) Oval

Q3 Best type for a price such as 1499.90 TL?

- A) int
- B) float
- C) decimal
- D) char

Q4 Whose body always runs at least once?

- A) for
- B) while
- C) do-while
- D) none

ANSWERS

Quick-Fire Multiple Choice: Answers

Q1 "Keep adding 1 to x." Which property fails?

- A) Definiteness
- **B) Finiteness**
- C) Input
- D) Output

There is no stopping condition, so it never ends: finiteness.

Q2 Which flowchart symbol is used for READ x?

- A) Rectangle
- B) Diamond
- **C) Parallelogram**
- D) Oval

Input and output both use the parallelogram.

Q3 Best type for a price such as 1499.90 TL?

- A) int
- B) float
- **C) decimal**
- D) char

decimal stores decimal digits exactly; use it for money.

Q4 Whose body always runs at least once?

- A) for
- B) while
- **C) do-while**
- D) none

do-while tests its condition after the body (post-test).

PREDICT · 4 min

Predict the Output: Eight Lines

Program.cs

```
 1  int a = 17, b = 5;
 2  Console.WriteLine(a / b);
 3  Console.WriteLine(a % b);
 4  Console.WriteLine(a / (double)b);
 5  Console.WriteLine(a + b * 2 - 1);
 6  Console.WriteLine(-a % b);
 7  Console.WriteLine("Sum: " + a + b);
 8  Console.WriteLine(a > b && b != 5);
 9  int x = 5;
10  int y = x++ * 2;
11  Console.WriteLine($"{x} {y}");
```

Your answer

1 _____

2 _____

3 _____

4 _____

5 _____

6 _____

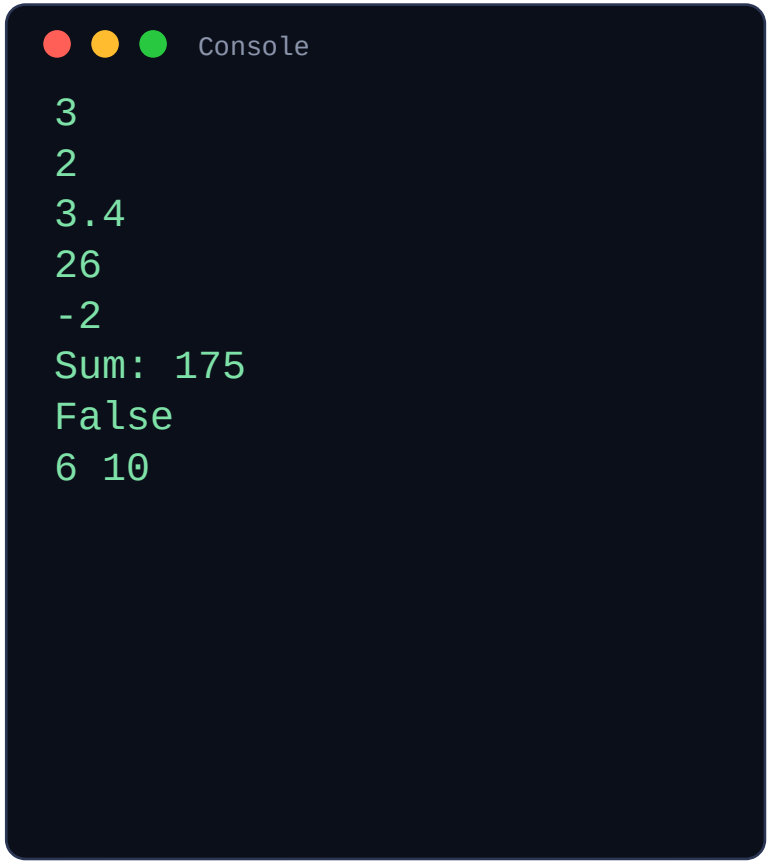
7 _____

8 _____

ANSWER

Predict the Output: Answer

#	Output	Why
1	`3`	int / int is integer division
2	`2`	17 = 3 · 5 + 2
3	`3.4`	one double operand → real division
4	`26`	* first: 17 + 10 − 1
5	`-2`	% keeps the sign of the left operand
6	`Sum: 175`	string + int → string, left to right
7	`False`	true && false; C# prints False
8	`6 10`	x++ gives 5 to the expression, then x = 6



Real output of the code (08_01_PredictChecker).

TRACE · 5 min

Trace the while Loop

```
1  int n = 1230, rev = 0, count = 0;
2  while (n > 0)
3  {
4      int d = n % 10;
5      rev = rev * 10 + d;
6      n = n / 10;
7      count++;
8  }
9  Console.WriteLine($"{rev} {count}");
```

Pass	d	rev	n	count
start	–	0	1230	0
1				
2				
3				
4				
Output				

? How many times is n > 0 tested?

ANSWER

Trace the while Loop: Answer

```
1  int n = 1230, rev = 0, count = 0;
2  while (n > 0)
3  {
4      int d = n % 10;
5      rev = rev * 10 + d;
6      n = n / 10;
7      count++;
8  }
9  Console.WriteLine($"{rev} {count}");
```

Pass	d	rev	n	count
start	–	0	1230	0
1	0	0	123	1
2	3	3	12	2
3	2	32	1	3
4	1	321	0	4
Output	**321 4**			

n > 0 is tested 5 times (4 × true, then false when n = 0). The trailing 0 of 1230 becomes a leading 0: 321, not 0321.

PREDICT · 3 min

Predict the Output: Decisions

(a)

```
int score = 85;
if (score >= 50)
{
    Console.WriteLine("DD");
}
else if (score >= 85)
{
    Console.WriteLine("BA");
}
else
{
    Console.WriteLine("FF");
}
```

(b)

```
int day = 7;
switch (day)
{
    case 6:
    case 7:
        Console.WriteLine("Weekend");
        break;
    default:
        Console.WriteLine("Weekday");
        break;
}
```

(c)

```
// (c) same score
Console.WriteLine(
  score % 2 == 0
  ? "even" : "odd");
```

? Is the "BA" line reachable for any score?

ANSWER

Predict the Output: Decisions, Answer

(a)

```
DD
```

(b)

```
Weekend
```

(c)

```
odd
```

(a) Order!

Only the FIRST true branch runs. 85 ≥ 50 already, so "BA" is never reached, for any score. Put ≥ 85 before ≥ 50.

(b) Grouped labels

case 6: and case 7: share one body. No fall-through in C#: each body ends with break.

(c) ?: operator

85 % 2 is 1, the condition is false, so the second value "odd" is chosen.

```
if (score >= 85) { Console.WriteLine("BA"); }       // fix: most restrictive test first
else if (score >= 50) { Console.WriteLine("DD"); } else { Console.WriteLine("FF"); }
```

PREDICT · 4 min

Predict the Output: Nested for

```
1  for (int i = 1; i <= 4; i++)
2  {
3      for (int j = 1; j <= 4 - i; j++)
4      {
5          Console.Write(" ");
6      }
7      for (int k = 1; k <= 2 * i - 1; k++)
8      {
9          Console.Write("*");
10     }
11     Console.WriteLine();
12 }
```

(a) Draw the output on a grid, spaces included.

(b) How many times does Console.Write("*") run?

grid: 4 rows × 8 columns

ANSWER

Predict the Output: Nested for, Answer



i	spaces (4 − i)	stars (2i − 1)
1	3	1
2	2	3
3	1	5
4	0	7
Total	6	**16**

(b) 16 times. 1 + 3 + 5 + 7 = 16 = 4^2. The outer loop runs 4 times; the inner loops run a different number of times in each pass.

Method: never simulate nested loops in your head. Make a table with one row per outer pass.

PREDICT · 4 min

Predict the Output: break, continue, do-while

(a)

```
1  int total = 0;
2  for (int i = 1; i <= 10; i++)
3  {
4      if (i % 3 == 0)
5      {
6          continue;
7      }
8      if (i > 7)
9      {
10         break;
11     }
12     total += i;
13 }
14 Console.WriteLine(total);
```

(b)

```
1  int k = 10;
2  do
3  {
4      Console.Write(k + " ");
5      k += 5;
6  } while (k < 10);
7  Console.WriteLine($"| k = {k}");
```

?

Would the output of (b) change if do-while were replaced by while (k < 10) { … }?

ANSWER

break, continue, do-while: Answer

i	1	2	3	4	5	6	7	8
action	+1	+2	skip	+4	+5	skip	+7	break
total	1	3	3	7	12	12	19	19

(a)

```
19
```

(b) do-while

```
10 | k = 15
```

(b) with while instead

```
| k = 10
```

continue vs break

continue skips the rest of THIS pass; break leaves the loop completely.

Post-test

do-while runs the body once before testing; while may run it zero times.



PART 3

Practice II: Debug, Draw, Write

Find the bugs · draw the flowchart · write the program · exam strategy

FIND BUGS · 5 min

Find the Bugs: Average of n Grades

Intended: read n grades, print their average

```
1  Console.Write("How many grades? ");
2  int n = int.Parse(Console.ReadLine());
3  int sum = 0;
4  for (int i = 1; i < n; i++);
5  {
6      Console.Write("Grade: ");
7      int g = int.Parse(Console.ReadLine());
8      sum += g;
9  }
10 double avg = sum / n;
11 Console.WriteLine("Average: {avg:F2}");
```

5 problems

Four lines are wrong and one check is missing. It compiles (with 1 warning).

Warm-up

What does the program print for n = 3 and grades 70, 80, 90?

ANSWER

Find the Bugs: Answer

Line	Bug	Fix
4	i < n reads only n − 1 values (off by one)	`i <= n`
4	; after the for header: empty loop body	`delete the ;`
10	sum / n is integer division	`(double)sum / n`
11	no $, so {avg:F2} is printed as text	`$"Average: {avg:F2}"`
2	n = 0 → DivideByZeroException	`check n > 0 first`

Compiler warning CS0642 "Possible mistaken empty statement" points at line 4. Warnings are hints: read them.

Buggy version, real run

```
How many grades? 3
Grade: 70
Average: {avg:F2}
```

Only one grade is read: the loop body is empty (;), the block after it runs once.

FLOWCHART · 6 min

Draw the Flowchart: Sum of Even Numbers

Problem. Read a whole number N. Compute the sum of all even numbers from 1 to N and print it. Example: N = 10 → 2 + 4 + 6 + 8 + 10 = 30.

Use these symbols

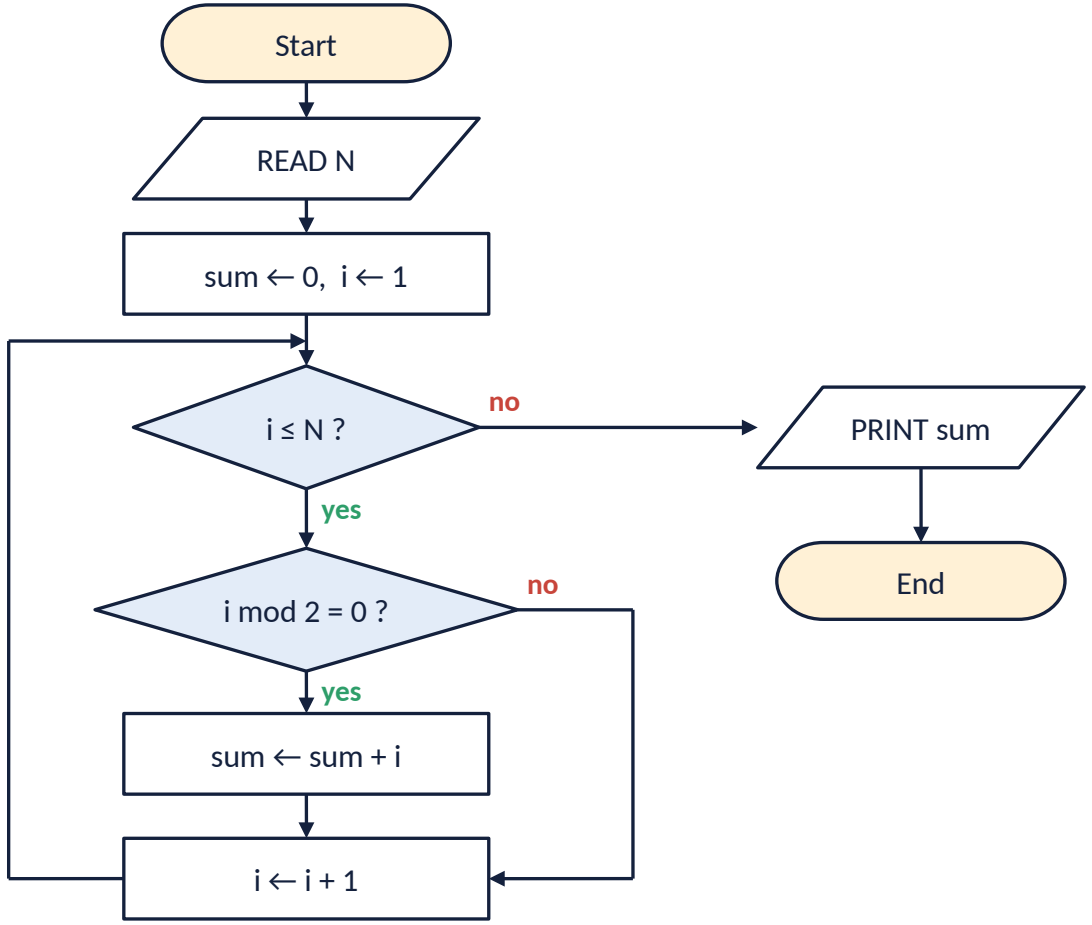


Checklist: 2 decisions (loop test, even test) · counter update · loop back-arrow · both exits labelled.

Sketch your flowchart here

ANSWER

Sum of Even Numbers: Flowchart and C#




```
08_03_SumOfEvens
Console.Write("N = ");
int n = int.Parse(Console.ReadLine());
int sum = 0;
for (int i = 1; i <= n; i++)
{
    if (i % 2 == 0)
    {
        sum += i;
    }
}
Console.WriteLine(
    $"Sum of even numbers from 1 to {n}: {sum}");
```

```
Console
N = 10
Sum of even numbers from 1 to 10: 30
```

WRITE CODE · 10 min

Write the Program: Grade Statistics

Specification

1. Read integer grades until the user types -1.
2. A grade outside 0–100 prints "Invalid grade, ignored." and is not counted.
3. At the end print: count, average (2 decimals), highest grade, and how many passed (≥ 50).
4. If no valid grade was entered, print "No grades entered."
5. Use only while/if (no arrays).

Required sample run

```
Grade (-1 to stop): 72
Grade (-1 to stop): 45
Grade (-1 to stop): 105
  Invalid grade, ignored.
Grade (-1 to stop): 90
Grade (-1 to stop): 58
Grade (-1 to stop): -1
Count: 4
Average: 66.25
Highest: 90
Passed: 3 of 4
```

ANSWER

Grade Statistics: Design and the Loop

Pseudocode

```
START
  count ← 0, sum ← 0
  highest ← 0, passed ← 0
  READ grade
  WHILE grade ≠ -1
    IF grade < 0 OR grade > 100 THEN
      PRINT "Invalid"
    ELSE
      count ← count + 1
      sum ← sum + grade
      IF count = 1 OR
         grade > highest THEN
        highest ← grade
      END IF
      IF grade ≥ 50 THEN
        passed ← passed + 1
      END IF
    END IF
    READ grade
  END WHILE
```

C# (part 1, braces compressed to fit)

```
int count = 0, sum = 0, highest = 0, passed = 0;
Console.Write("Grade (-1 to stop): ");
int grade = int.Parse(Console.ReadLine());
while (grade != -1)
{
    if (grade < 0 || grade > 100)
    {
        Console.WriteLine("  Invalid grade, ignored.");
    }
    else
    {
        count++;
        sum += grade;
        if (count == 1 || grade > highest) { highest = grade; }
        if (grade >= 50) { passed++; }
    }
    Console.Write("Grade (-1 to stop): ");
    grade = int.Parse(Console.ReadLine());
}
```

ANSWER

Grade Statistics: Results and Testing

C# (part 2)

```
if (count == 0)
{
    Console.WriteLine("No grades entered.");
}
else
{
    double average = (double)sum / count;
    Console.WriteLine($"Count: {count}");
    Console.WriteLine($"Average: {average:F2}");
    Console.WriteLine($"Highest: {highest}");
    Console.WriteLine($"Passed: {passed} of {count}");
}
```

Edge case: sentinel first

```
Grade (-1 to stop): -1
No grades entered.
```

Test input	Count · Average · Highest · Passed
72 45 105 90 58 -1	4 · 66.25 · 90 · 3
-1	No grades entered.
30 40 -1	2 · 35.00 · 40 · 0
50 -1	1 · 50.00 · 50 · 1

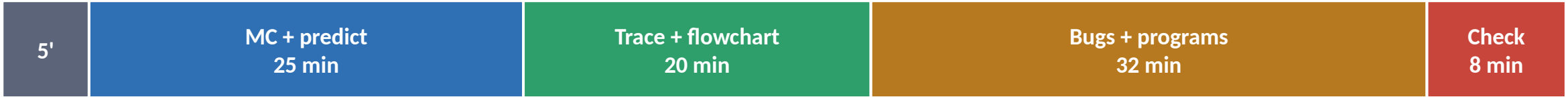
Test normal data, an invalid value, the sentinel first, and a boundary value (exactly 50).

Ten Mistakes That Cost Points

#	Wrong	Right	Why
1	`if (x = 5)`	`if (x == 5)`	= assigns, == compares
2	`int n = Console.ReadLine();`	`int n = int.Parse(Console.ReadLine());`	ReadLine returns a string
3	`decimal p = 9.99;`	`decimal p = 9.99m;`	literal type must match
4	`a / b  with b = 0 (int)`	`if (b != 0) …`	DivideByZeroException
5	`avg = (a + b) / 2;  (ints)`	`avg = (a + b) / 2.0;`	integer division
6	`if (0 <= x <= 100)`	`if (x >= 0 && x <= 100)`	no chained comparisons
7	`for (…);  /  while (…);`	`no ; after the header`	empty loop body
8	`for (int i = 1; i < n; i++)`	`for (int i = 1; i <= n; i++)`	off-by-one: n passes
9	`while loop without i++`	`update the loop variable`	infinite loop
10	`max = 0; (negative data)`	`max = first value`	best-so-far pattern

>_ EXAM STRATEGY

A 90-Minute Plan and a Method per Question Type



Read all: 5 min

Predict / trace

Make a table: one column per variable. Update one line at a time. Write spaces as ␣ in patterns.

Flowchart

Pencil first. Standard symbols, arrows, yes/no on every decision, one End.

Find the bug

Check each line for: ; placement, = vs ==, types, division, loop bounds, missing $.

Write a program

IPO → pattern → pseudocode → C#. Prompts, Parse/TryParse, then the logic, then the output format.

Never leave a question blank: a correct plan, pseudocode or a partial trace usually earns partial credit.

Your Study Plan for the Midterm



Day 1

Re-read the cheat sheet in the notes. Write your own one-page summary by hand.



Day 2

Take the practice midterm in the notes: 90 minutes, closed book, on paper.



Day 3

Check your answers (run the code folder). List every mistake and its cause.



Day 4

Problem set P1–P12. Type your programs, compare with the code folder.



Day 5

Explain two solutions to a classmate. Bring open questions to class or office hours.



Code folder: 14 tested projects (08_01 … 08_14): every programming question of this deck, the practice exam and the problem set.

>_ SUMMARY

Key Takeaways

01 The midterm covers Weeks 1–7: algorithms, flowcharts, types, operators, decisions, loops.

02 Trace with a table: one column per variable, one row per pass.

03 Watch integer division, % signs, operator precedence and string + number.

04 Only the first true branch of an else-if ladder runs; order the conditions.

05 Choose the loop: for = count known, while = condition, do-while = at least once.

06 Name the pattern (accumulator, best so far, sentinel, validation, digits), then code it.

NEXT WEEK

Midterm exam · then Week 9: One-Dimensional Arrays



MCMP113
Course materials